Transfer Learning & the DocScan Capstone

Week 16 — Session 2: Shipping the project

Practical Image Processing

Lecture 16.2 ▪ 60–70 min

Recap of Session 1

- Small data (180 images) $\rightarrow$ can't train a big CNN from scratch.
- **Transfer learning**: freeze a pretrained net, use it as a feature extractor.
- Same SVM, deep features instead of HOG: **0.48** $\rightarrow$ **0.87**.
- Augmentation + the backbone's own preprocessing squeeze out the rest.

Today

We can read digits. Now assemble the **whole DocScan pipeline** and engineer it to survive the real world — then ship the capstone.

Roadmap

Fine-tuning: the deeper option

The DocScan capstone

Engineering for the real world

In-class exercise

Course wrap-up

The deliverable

Fine-tuning: the deeper option

When feature extraction isn't enough

- Feature extraction keeps the backbone *fixed* — great for tiny data.
- With more data (thousands of images) you can **fine-tune**: let some top layers adapt to your domain.
- This adjusts mid/late features toward *your* task, not ImageNet's 1000 classes.

Key idea

Fine-tuning trades data and compute for a few extra points of accuracy — spend it only when you have the data.

The fine-tuning recipe

1. Start from the pretrained weights (never random).
2. Replace the classifier head for your classes.
3. **Freeze** early layers; unfreeze the top k.
4. Train with a *small* learning rate (10^{-4} or less).
5. Stop early — watch the validation curve.

Why a small LR

The pretrained weights are already good. A large step

$$w \leftarrow w - \eta \nabla$$

with big η would erase what the giant learned.

Pitfall

A big learning rate on a pretrained net is **catastrophic forgetting** — you destroy the transfer.

Freeze depth vs. data size

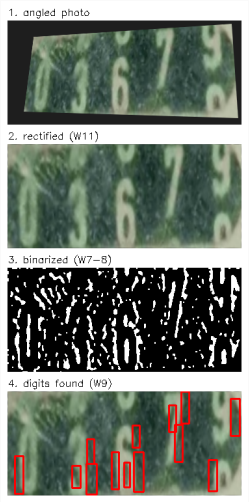
Data you have	Strategy	Unfreeze
hundreds	feature extraction	nothing (frozen)
a few thousand	fine-tune top block	last 1–2 stages
tens of thousands	fine-tune widely	most of the net
millions	train from scratch is viable	everything

✔ Good practice

Our DocScan digits use *feature extraction* — hundreds of samples, small domain gap.

The DocScan capstone

DocScan: the whole course in one pipeline



- **1. Photo** — a meter snapped at an angle (raw input).
- **2. Rectify** (W11) — homography warps it flat.
- **3. Binarize** (W2/7/8) — threshold + morphology clean it.
- **4. Segment** (W9) — contours find each digit box.
- **5. Read** (W13) — a CNN classifies each digit → “03679”.

Every week was a stage

Weeks	DocScan role
1–4 Pixels, histograms, filtering, denoise	clean the raw photo
5–6 Edges, Fourier	find structure, remove periodic noise
7–9 Morphology, segmentation, contours	isolate the digits
10–11 Features, homography	rectify to a flat scan
12–16 ML, CNNs, transfer learning	read the digits

Key idea

DocScan isn't a separate project — it's the spine that every week added a vertebra to.

The pipeline as one function (code)

```python
def docscan(photo):
    warped = rectify(photo)                 # W11: homography + RANSAC
    if warped.inliers < 12:                 # confidence gate
        return Reject("blurry / bad angle - retake")
    binary = clean(warped.image)            # W2/7/8: threshold + morphology
    boxes  = segment_digits(binary)         # W9: contours -> ordered boxes
    if len(boxes) != EXPECTED_DIGITS:       # sanity check
        return Reject("wrong digit count")
    digits = [read_digit(b) for b in boxes] # W13 / transfer features
    return "".join(d.label for d in digits)
```

- Each helper is a week of the course; the glue is engineering.

Ordering the digit boxes

- Contours come back in *arbitrary* order — you must sort left-to-right.
- After rectification, sort boxes by their center x.
- Guard against split/merged blobs by checking width and spacing.



sorted by center-x $\rightarrow$ reading order “03679”

✕ Pitfall

Skip the sort and “03679” becomes “73096”. Ordering is not optional.

Engineering for the real world

Engineering a robust pipeline

- **Confidence gates**: if RANSAC inliers are too few, ask for a new photo (don't guess).
- **Sanity checks**: a meter reading has a fixed number of digits — reject wrong counts.
- **Fallbacks**: if the feature path fails, try the contour path (and vice-versa).
- **Logging**: record per-stage quality (RMSE, digit confidences) to debug failures.

✕ Pitfall

A demo works on one clean image; a product survives blur, glare, tilt, and dirt. Design for failure.

Confidence gating, quantified

Per-read confidence

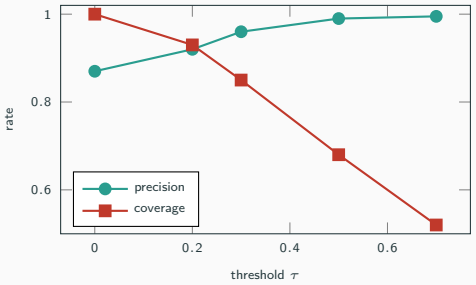
A softmax digit read gives $p_1 \geq p_2 \geq \ldots$. Gate on the *margin*:

$$m = p_1 - p_2$$

Accept only if $m > \tau$ (e.g. $\tau = 0.3$); else flag for review.

- High p_1 but small margin = the net is torn between two digits.
- One low-margin digit taints the whole reading — gate per digit.
- Raising τ trades *coverage* for *precision*.

The precision / coverage trade-off

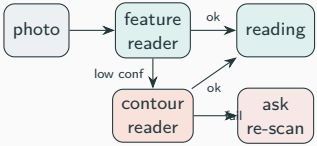


- Raise τ: fewer readings, but the ones you keep are more reliable.
- Billing wants *precision*; a triage tool may want *coverage*.
- Pick τ from the cost of a wrong bill vs. a re-scan.

✓ Good practice

State your operating point: "at $\tau = 0.3$, 96% precision over 85% of photos."

Fallbacks and graceful failure



- Two independent readers rarely fail on the *same* image.
- If both disagree or gate out, *refuse* — don't emit a guess.
- A refusal is cheap; a wrong bill is expensive and erodes trust.

In-class exercise

In-class exercise (10 min)

Design the system

1. In DocScan, the rectify step returns very few RANSAC inliers. What should the system do?
2. Your digit reader outputs $p_1 = 0.95$, $p_2 = 0.90$ for one digit. Accept it? Why?
3. Segmentation returns 4 boxes but meters have 5 digits. What now?
4. You want to guarantee correct bills. Do you raise or lower the confidence threshold?

Debrief

- (1) **Gate it**: treat rectification as unreliable and prompt for a new photo rather than reading a bad scan.
- (2) **No** — the margin $m = 0.05$ is tiny; the net is unsure between two digits. Flag for review.
- (3) **Reject / re-segment**: the digit-count sanity check failed; don't emit a 4-digit reading.
- (4) **Raise** τ — higher precision, fewer but more trustworthy readings.

Key idea

Honest *failure handling* beats a fancier model — a system that knows when it doesn't know is worth shipping.

A latency budget

Stage	Time (ms)
rectify (homography)	8
clean + morphology	5
segment contours	4
feature extraction (backbone)	45
SVM classify $\times 5$ digits	3
total	$\approx$**65**

- The *backbone* dominates — that's where transfer's cost lives.
- On-device, swap to a smaller backbone or quantize to hit a budget.
- Measure before optimizing; the contour path is nearly free.

✓ Good practice

Profile per stage. You can't shrink what you never timed.

Common capstone pitfalls

- **Preprocessing mismatch**: your resize/normalize differs from the backbone's — accuracy silently drops.
- **Data leakage**: same meter appears in train and test, or the scaler saw test data.
- **Unsorted digits**: correct classes, wrong order — a subtle, common bug.
- **No test number**: a demo on one image is not an evaluation.

✕ Pitfall

Each of these can turn a real 0.87 into a fake 0.99 or a mysterious 0.40. Check them first.

Course wrap-up

The arc of the course

- We started with a single pixel and ended with a pretrained neural network.
- Classical IP (weeks 1–11): precise, interpretable, no training — still essential for geometry & cleanup.
- Learning (weeks 12–16): powerful for recognition, especially via transfer learning.
- A real system uses both: classical to prepare, learned to recognize.

Choosing the right tool

If you need to...	Reach for...
align/rectify/measure geometry	classical (homography, contours)
clean noise, threshold, morphology	classical filters
recognize with small data	transfer learning
recognize with big data	train/fine-tune a CNN

Key idea

The mark of an engineer: picking the simplest tool that solves the problem reliably.

Honesty, ethics, and limits

- Report the *test* number, the failure cases, and the data you trained on — not just the best demo.
- A misread meter digit is a wrong bill; know your error rate and gate low-confidence reads.
- Beware dataset bias: if all training meters are clean and bright, expect failure on dirty ones.

✕ Pitfall

"97% accurate" means nothing without: on what data, which metric, and how it fails.

A quick self-check on the whole course

- Can you clean a noisy image (denoise, threshold, morphology)? (W1–8)
- Can you find and measure shapes/contours? (W9)
- Can you rectify a tilted photo with a homography? (W10–11)
- Can you build a features→classifier and a CNN, and use transfer learning? (W12–16)

Key idea

If yes to all four, you can build the majority of practical vision systems end to end.

Where to go next

- **Detection** (YOLO, Faster-RCNN): find *and* locate objects — the next step past classification.
- **Segmentation** (U-Net, SAM): label every pixel — for your power-line/mask datasets.
- **OCR** systems (Tesseract, TrOCR): full text reading pipelines built on these ideas.
- **Deployment**: on-device (TFLite, ONNX), latency, and robustness in the field.

The deliverable

The capstone deliverable

Final project (see `capstone/`)

1. Assemble the full DocScan pipeline on real `E:/data` images.
2. Rectify (W11) $\rightarrow$ clean/segment (W7–9) $\rightarrow$ read digits (W13 or transfer).
3. Add at least two robustness features (confidence gate, digit-count check, fallback).
4. Report end-to-end accuracy and where it fails; propose one concrete improvement.

How your capstone is judged

Criterion	Weight
Working end-to-end pipeline	40%
Robustness features (2+)	20%
Honest evaluation & failure analysis	25%
Clarity of report & code	15%

✓ Good practice

A modest pipeline with an honest error analysis beats a flashy demo with no test number.

✗ Pitfall

No confidence gate + one cherry-picked image = an automatic re-do.

A reproducibility checklist

✔ Before you submit

- Fixed random seed; train/test split saved.
- Backbone + preprocessing pinned to versions.
- One command reruns the whole pipeline.

- A result nobody can rerun is a claim, not evidence.
- Log the test number, the threshold τ, and the failure cases together.
- Future-you (and your grader) should get the same number twice.

Key idea

If it doesn't reproduce, it didn't happen. Ship the recipe, not just the result.

From one pixel to a working scanner.

Classical vision to prepare, deep learning to recognize — and the judgment to know which, when.